

Teleport S3 Parquet to Coralogix

Architecture, Lambda image, IAM, and Terraform deployment

Package: container image

Architecture: x86_64 / linux/amd64

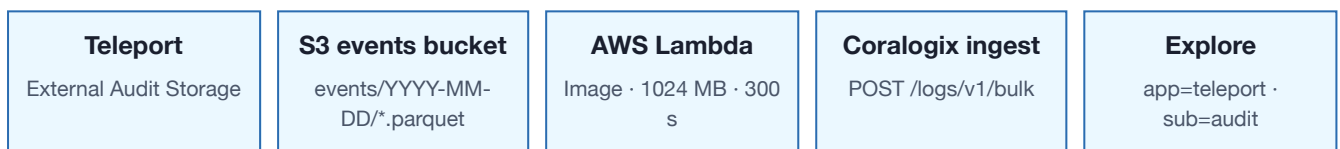
No Lambda layers

This integration is for Teleport clusters that already use **External Audit Storage / Athena** and land audit events as Snappy-compressed parquet in S3. An AWS Lambda function reads each new object and POSTs every event to the Coralogix Logs API.

Terraform does **not** create the Teleport cluster or the events bucket. Those already exist. The module creates ECR (unless you supply an image), the Lambda function, the execution role, and an optional S3 notification.

Do not run in parallel with Teleport Event Handler / OpenTelemetry export unless you want duplicate events in Coralogix.

1. End-to-end architecture



write parquet → ObjectCreated → parse & ship → query by parquet event_time

```

s3://<events-bucket>/events/YYYY-MM-DD/<worker>-<timestamp>.parquet
|
| s3:ObjectCreated:* prefix events/ suffix .parquet
v
AWS Lambda (package_type = Image, architectures = x86_64)
|
| POST https://ingress.<CORALOGIX_DOMAIN>/logs/v1/bulk
| applicationName=teleport subsystemName=audit
v
Coralogix Logs → Explore

```

The S3 trigger fires only for **new** objects. Existing files need a backfill invoke. The handler also unwraps SNS or SQS that wrap the same S3 JSON, and accepts a direct payload:

```
{"bucket": "your-events-bucket", "key": "events/2026-09-11/file.parquet"}
```

2. Why a container image (not zip, not layers)

Teleport parquet is Snappy-compressed. Reading it in Python requires **pyarrow**. That library is too large for a typical Lambda zip package (50 MB uploaded / 250 MB unzipped). A Lambda **layer** has the same unzipped size cap and must still match Python 3.12 and x86_64. This integration therefore uses a **container image only** and attaches **no layers**. Dependencies are installed inside the image.

Option	Limit / issue	Verdict
Lambda zip	50 MB uploaded, 250 MB unzipped; pyarrow wheel is ~50 MB	Not reliable
Lambda layer	Same unzipped cap; extra build that must match runtime/arch	Do not use
Container image	Up to 10 GB; <code>pip install</code> on the official AWS base	Required

3. Image that is required

Use the official AWS Lambda Python 3.12 base. This image includes the Lambda Runtime Interface Client (RIC). The function CMD must be `lambda_function.lambda_handler`.

```
FROM public.ecr.aws/lambda/python:3.12
COPY requirements.txt ${LAMBDA_TASK_ROOT}/
RUN pip install --no-cache-dir -r requirements.txt
COPY lambda_function.py ${LAMBDA_TASK_ROOT}/
CMD ["lambda_function.lambda_handler"]
```

Requirement	Value	Why
Base image	<code>public.ecr.aws/lambda/python:3.12</code>	Official RIC + Python 3.12. Generic Python or Alpine images are not Lambda-compatible.
OS / CPU	<code>linux/amd64</code> (<code>x86_64</code>)	Terraform sets <code>architectures = ["x86_64"]</code> . An arm64 / Apple-silicon-only image will not run.
Manifest	Single image, not a Docker index	Lambda rejects attestation/SBOM indexes: "The image manifest ... is not supported".
Build flags	<code>--platform linux/amd64 --provenance=false --sbom=false --push</code>	Required on Apple silicon and on Docker Desktop (attestations are on by default).
Python packages	<code>pyarrow>=17</code> , <code>boto3>=1.34</code>	Snappy parquet + S3 GetObject. No other OS packages.
Registry	ECR, same account and region as Lambda	Default repo/tag: <code>teleport-s3-parquet-coralogix:latest</code>

Image filesystem (Docker layers)

Layer	Contents	Purpose
1. AWS base	Amazon Linux, Python 3.12, Lambda RIC	What the Lambda service boots
2. pip install	pyarrow, boto3	Read parquet; call S3. No Lambda layers attached.
3. Application	<code>lambda_function.py</code>	Handler only

4. Lambda architecture (the function)

Setting	Default	Notes
Package type	Image	Not Zip
Architecture	x86_64	Must match the image
Memory	1024 MB	Use 2048 MB for large parquet files (also more CPU)
Timeout	300 seconds	A single file can contain many thousands of events
VPC	None	Lambda needs egress to <code>ingress.<domain></code>
Region	Same as events bucket	S3 notifications cannot target a function in another region

Handler flow

1. Extract bucket and key from S3, SNS, SQS, or a direct JSON payload.
2. Skip if the suffix is not `.parquet`, the prefix does not match `events/`, or the path contains `/sessions/`.
3. `s3.get_object` then `pyarrow.parquet.read_table`.
4. Map columns `uid`, `session_id`, `event_type`, `user`, `event_time`, `event_data` (JSON — the full Teleport audit event).
5. POST batches of 400 to Coralogix; retry on HTTP 429 and 5xx.

Environment variables

Variable	Required	Default	Description
CORALOGIX_SEND_YOUR_DATA_KEY	Yes	—	Send-Your-Data API key (prefer <code>TF_VAR</code> , not a file)
CORALOGIX_DOMAIN	Yes	coralogix.com	Team domain only, e.g. eu1.coralogix.com — not ingress.
CORALOGIX_APPLICATION_NAME	No	teleport	Coralogix application
CORALOGIX_SUBSYSTEM_NAME	No	audit	Coralogix subsystem
S3_KEY_PREFIX	No	empty / <code>events/</code>	Ignore session recordings
S3_KEY_SUFFIX	No	<code>.parquet</code>	Object suffix
CORALOGIX_BATCH_SIZE	No	400	Events per HTTP request
DRY_RUN	No	false	Parse only, do not POST

5. What Terraform creates — and what it does not

Created	Purpose	Not created
ECR repository	Holds the linux/amd64 image (unless image_uri is set)	Teleport cluster
IAM role + inline policy	GetObject on events/*, ListBucket, CloudWatch Logs, optional KMS	Events S3 bucket
Lambda (Image)	Runs the handler on each new parquet object	Session-recording pipeline
lambda:AddPermission + optional S3 notification	s3.amazonaws.com may InvokeFunction	Coralogix parsers

S3 notification replace semantics. `create_s3_notification = true` writes a new bucket notification configuration and *replaces* any existing ones. Set it to `false` and add the trigger in the console if the Teleport bucket already notifies other systems.

`manage_log_group = false` (default) lets Lambda create `/aws/lambda/<name>` on first invoke. Set true only if the deployer can call `logs:DescribeLogGroups` on `*`.

6. IAM: deployer vs runtime

Deployer (human or CI)

Uses `terraform/deployer-iam-policy.json`.
Replace `AWS_REGION`, `AWS_ACCOUNT_ID`, and `EVENTS_BUCKET`.

- ECR create / login / push
- IAM CreateRole / PassRole
- Lambda create / update / invoke
- S3 GetBucketNotification / PutBucketNotification
- Optional DescribeLogGroups on `*`

Runtime role (the function)

Created by Terraform as `<function_name>-role`.
Least privilege at invoke time only.

- `s3:GetObject` on `events/*`
- `s3:ListBucket` with prefix
- CloudWatch Logs write
- Optional `kms:Decrypt` if the bucket is SSE-KMS

7. Prerequisites

Item	Notes
Terraform <code>>= 1.5</code>	<code>terraform version</code>
AWS CLI + credentials	Same account and region as the events bucket
Docker (or equivalent)	Must build linux/amd64, including on Apple silicon
Existing S3 events bucket	Teleport parquet, not session recordings
Coralogix Send-Your-Data key	Data Flow → API Keys
Coralogix domain	Must match the team: eu1.coralogix.com, eu2, us1, coralogix.in, ...

8. Deploy (recommended path)

```
cd "SIEM & SaaS/Teleport-S3-Parquet/terraform"
cp values.auto.tfvars.example values.auto.tfvars
# edit: aws_region, events_bucket, coralogix_domain, create_s3_notification

export AWS_PROFILE=your-profile
export TF_VAR_coralogix_api_key='your-send-your-data-key'

cd ..
chmod +x deploy.sh
./deploy.sh
```

`deploy.sh` runs `terraform init`, creates ECR if needed, builds and pushes the linux/amd64 image without attestations, then `terraform apply`.

Terraform variables

Name	Required	Default	Notes
events_bucket	Yes	—	Existing Teleport events bucket
coralogix_api_key	Yes	—	Export TF_VAR_coralogix_api_key — do not commit
aws_region	No	eu-west-1	Must match the bucket region
coralogix_domain	No	eu1.coralogix.com	Team domain, not ingress.
events_prefix / events_suffix	No	events/ · .parquet	Change if keys use a tenant prefix
create_s3_notification	No	true	Replaces all bucket notifications
manage_log_group	No	false	Needs DescribeLogGroups if true
image_uri	No	empty	Set to skip ECR create + build
kms_key_arn	No	empty	SSE-KMS buckets

9. Verify and find logs

Success payload: `{"ok":true,"files":1,"events":N,"sent":N}`.

```
aws s3 cp ./sample.parquet s3://YOUR-BUCKET/events/$(date -u +%Y-%m-%d)/sample.parquet

aws lambda invoke \
  --function-name teleport-s3-parquet-coralogix \
  --cli-binary-format raw-in-base64-out \
  --payload '{"bucket":"YOUR-BUCKET","key":"events/YYYY-MM-DD/sample.parquet"}' \
  /tmp/out.json && cat /tmp/out.json
```

Explore time range. Use the *event time stored in the parquet* (`event_time / time`), not “last 15 minutes”, unless the file is new. Query: `applicationName:teleport AND subsystemName:audit`.

Backfill existing objects

```
aws s3 ls s3://YOUR-BUCKET/events/ --recursive \
| awk '{print $4}' | grep '\.parquet$' \
| while read -r key; do
    aws lambda invoke --function-name teleport-s3-parquet-coralogix \
        --cli-binary-format raw-in-base64-out \
        --payload "{\"bucket\":\"YOUR-BUCKET\",\"key\":\"${key}\"}" /tmp/out.json
done
```

10. Troubleshooting

Symptom	What to check
Image manifest is not supported	Rebuild with --provenance=false --sbom=false and linux/amd64
Function never runs	Prefix/suffix, lambda:AddPermission, bucket region vs aws_region
AccessDenied on GetObject	Runtime role + kms:Decrypt if SSE-KMS
logs:DescribeLogGroups AccessDenied	Keep manage_log_group = false
Empty Explore	Wrong team/domain, or time range is not parquet event_time
Timeout	Raise memory (and CPU) and timeout
Duplicate events	Disable Event Handler if this S3 path is the only source

Source: snowbit-integrations / SIEM & SaaS / Teleport-S3-Parquet. Destroy with `terraform destroy` using the same variables; the events bucket is never deleted. ECR uses `force_delete`. Do not commit API keys or tfstate.